multiSpeciesRegionFoam

Physics Reference Guide

Species transport through solid membranes and porous materials
in the OpenFOAM-13 `foamMultiRun` framework

Scope. This document describes every physical model implemented in the `multiSpeciesRegionFoam` library: governing equations, material constitutive laws, interface and surface boundary conditions, performance metrics, and numerical discretisation. Each section indicates which tutorial cases (case311–case330) demonstrate the corresponding physics.

Jaydeep Deshpande

Contents

Introduction and Scope

`multiSpeciesRegionFoam` is an OpenFOAM-13 add-on that extends the native `foamMultiRun` [7] multi-region framework with coupled species transport. Two solver modules are provided:

- **`speciesSolid`** — adds a species conservation equation to solid (or fluid-region-with-prescribed-flow) regions alongside the inherited thermal energy equation from the `solid` base module.
- **`speciesFluid`** — extends `incompressibleFluid` with a solved temperature field and species transport equation. The full Navier–Stokes PIMPLE loop (U, p) is inherited; temperature and species are added in `thermophysicalPredictor()`.
- **`compressibleSpeciesFluid`** — extends the OpenFOAM-13 `fluid` module (compressible PIMPLE with full internal-energy equation) with species transport. Required when density varies with pressure, or when the standard OpenFOAM `heRhoThermo` property infrastructure is preferred (e.g. for gases or using `rhoConst` for liquids with thermophysical transport).

Inter-region coupling (heat and species) uses OpenFOAM's mapped-patch infrastructure, and all new boundary conditions are registered in the standard run-time selection tables [1].

Physical scope. The library covers the full range from pure-diffusion permeation barriers to coupled advection-diffusion in molten-salt channels. Typical applications are:

- Hydrogen / tritium permeation through first-wall and breeding-blanket materials in fusion machines; structural materials in molten salt reactors (Sieverts-law solubility).
- Dense-film membrane separation: reverse osmosis, membrane distillation, pervaporation (Henry-law solubility, Arrhenius temperature dependence).
- Permeation barriers (tungsten carbide, alumina coatings) [4].
- Heat-exchanger tritium transport: FLiBe/Hitec molten-salt systems with conjugate heat transfer and dissolved-species permeation.
- Fluid-channel species accumulation and wall permeation (speciesFluid).

Governing Equations

Species conservation

In each solid region the *mobile* species concentration $C\,[\mathrm{mol\ m^{-3}}]$ satisfies:

Species equation — every solid region

$$\frac{\partial C}{\partial t} = \nabla\cdot\left(D(T)\,\nabla C\right) - \frac{\partial C_t}{\partial t} + S_{\mathrm{vol}} \tag{1}$$

Symbol	Meaning	Units
C	Mobile species concentration	$\mathrm{mol\ m^{-3}}$
$D(T)$	Temperature-dependent diffusivity	$\mathrm{m^2\ s^{-1}}$
C_t	Trapped concentration (§3.3)	$\mathrm{mol\ m^{-3}}$
S_{vol}	Uniform volumetric source	$\mathrm{mol\ m^{-3}\ s^{-1}}$

When trapping is disabled (`trappingModel noTrapping`), $\partial C_t/\partial t = 0$ and the equation reduces to pure diffusion. The source term S_{vol} is optional and uniform within each region.

Demonstrated by: all twenty tutorial cases (`case311`–`case330`). Cases `case311`–`case312` use the thermal field T as a species proxy to verify the pure-diffusion solver before species coupling is introduced.

Energy conservation

The thermal energy equation is inherited from OpenFOAM-13's `solid` module:

Energy equation (Fourier conduction)

$$\frac{\partial(\rho\, e)}{\partial t} + \nabla \cdot \mathbf{q} = S_e, \qquad \mathbf{q} = -\kappa(T)\,\nabla T \tag{2}$$

Symbol	Meaning	Units
ρ	Density	kg m^{-3}
e	Specific internal energy	J kg^{-1}
$\mathbf{q}$	Conductive heat flux	W m^{-2}
$\kappa(T)$	Thermal conductivity	W m^{-1} K^{-1}
S_e	Volumetric heat source	W m^{-3}

The species and energy equations are solved *sequentially* within each PIMPLE outer corrector. After the energy equation updates T, the diffusivity $D(T)$ (and all Arrhenius properties) are refreshed before assembling the species equation.

Full energy–species coupling: `case316` (membrane distillation with Arrhenius $D(T)$ driven by a temperature gradient), `case320` (three-region shell-and-tube heat exchanger; conjugate heat transfer through FLiBe–SS316–Hitec).
Isothermal (energy solved but T fixed): `case311`–`case315`, `case317`–`case319`, `case321`–`case322`, `case328`.
Active T advection (`speciesFluid`): `case327`–`case328`.
Compressible energy eq. (`compressibleSpeciesFluid`): `case329`–`case330`.

Species and energy transport in fluid regions (`speciesFluid`)

The `speciesFluid` solver module targets incompressible fluid regions where the velocity field must be computed from the Navier–Stokes equations. It inherits the PIMPLE velocity-pressure loop from `incompressibleFluid` and adds temperature and species transport via an overridden `thermophysicalPredictor()` method.

Full equation set solved per PIMPLE outer corrector

Step 1 — Momentum and continuity (inherited, solved first):

Incompressible Navier–Stokes

$$\nabla \cdot \mathbf{U} = 0 \tag{3}$$

$$\frac{\partial \mathbf{U}}{\partial t} + \nabla \cdot (\mathbf{U}\mathbf{U}) = -\nabla p + \nabla \cdot (\nu\, \nabla \mathbf{U}) \tag{4}$$

where $p\,[\mathrm{m}^2\ \mathrm{s}^{-2}]$ is kinematic pressure and $\nu\,[\mathrm{m}^2\ \mathrm{s}^{-1}]$ is kinematic viscosity.

Step 2 — Temperature (scalar transport):

Energy equation — `speciesFluid` regions

$$\frac{\partial T}{\partial t} + \nabla \cdot (\mathbf{U}\, T) = \nabla \cdot (\alpha \nabla T), \qquad \alpha = \frac{\kappa}{\rho\, C_p} \tag{5}$$

This is the incompressible constant-property form of the energy equation, where $\alpha\,[\mathrm{m^2\ s^{-1}}]$ is the thermal diffusivity. It is solved as an advection-diffusion equation for T directly, rather than for the internal energy $e = C_p\, T$ used by the `solid` module.

Step 3 — Species concentration (advection-diffusion):

Species equation — `speciesFluid` regions

$$\frac{\partial C}{\partial t} + \nabla \cdot (\mathbf{U}\, C) = \nabla \cdot (D(T)\, \nabla C) - \frac{\partial C_t}{\partial t} + S_{\mathrm{vol}} \tag{6}$$

After solving T, the diffusivity $D(T)$ is refreshed via `species_.correctProperties()` before assembling (6). The trapping sink $\partial C_t/\partial t$ (§3.3) applies equally here; most fluid-region cases use `noTrapping`.

Numerical requirements for `speciesFluid`

The advection terms $\nabla \cdot (\mathbf{U}\, T)$ and $\nabla \cdot (\mathbf{U}\, C)$ produce *asymmetric* coefficient matrices. The following requirements must be observed in `fvSolution`:

Matrix solver: use `PBiCGStab` with `DILU` preconditioner for `"T.*"` and `"C_.*"`. `PCG` (symmetric-only) will abort with the error *"Unknown asymmetric matrix solver PCG"*.
Wildcard: use `"T.*"` (not bare `T`) in the `solvers` block. The PIMPLE final corrector creates a `TFinal` field; omitting the wildcard causes a *"keyword TFinal is undefined"* error.

Minimum `fvSolution` for a `speciesFluid` region:

```
solvers {
    "T.*"  { solver PBiCGStab; preconditioner DILU;
             tolerance 1e-10; relTol 0; }
    "C_.*" { solver PBiCGStab; preconditioner DILU;
             tolerance 1e-10; relTol 0; }
}
PIMPLE { nOuterCorrectors 3; nCorrectors 2;
         nNonOrthogonalCorrectors 0; }
```

Required `physicalProperties` format:

```
viscosityModel  Newtonian;           % read by incompressibleFluid
nu  [0 2 -1 0 0 0 0]  4.74e-7;
rho    983;                          % read by speciesFluid
Cp     4183;
mixture { transport { kappa  0.654; } }
```

Demonstrated by (`speciesFluid`): `case327` (DCMD with solved laminar flow; feed and permeate use `speciesFluid`), `case328` (Poiseuille + Graetz validation of the solver).
See also: `case329`–`case330` use `compressibleSpeciesFluid` (§2.4), which wraps the same PIMPLE loop via the compressible `fluid` base class.

Dimensional considerations — Schmidt and Péclet numbers

In fluid regions, two dimensionless groups govern the relative importance of advection and diffusion for momentum (Sc) and species (Pe):

$$\mathrm{Sc} = \frac{\nu}{D}, \qquad \mathrm{Pe} = \frac{U\,L}{D} = \mathrm{Re}\cdot\mathrm{Sc} \tag{7}$$

For liquid metals and molten salts, $\mathrm{Sc} \sim 10^2$–10^3 (molecular diffusivity is orders of magnitude smaller than momentum diffusivity). At high Pe the concentration boundary layer at a permeation wall is thin:

$$\frac{\delta_C}{L} \;\sim\; \mathrm{Pe}^{-1/3} \tag{8}$$

This sets the mesh resolution requirement near permeation walls. Upwind (`Gauss upwind`) or linearUpwind schemes for $\nabla\cdot(\mathbf{U}\,C)$ prevent numerical oscillations at high Pe.

Species transport in compressible fluid regions (`compressibleSpeciesFluid`)

The `compressibleSpeciesFluid` solver extends the OpenFOAM-13 `fluid` base module — which provides a compressible PIMPLE loop and full internal-energy equation — with single-species molar concentration transport.

Full equation set per PIMPLE outer corrector

Step 1 — Continuity + momentum + pressure (inherited from `isothermalFluid`/`fluid`):

Compressible continuity and momentum

$$\frac{\partial\rho}{\partial t} + \nabla\cdot(\rho\mathbf{U}) = 0 \tag{9}$$

$$\frac{\partial(\rho\mathbf{U})}{\partial t} + \nabla\cdot(\rho\mathbf{U}\mathbf{U}) = -\nabla p + \nabla\cdot\boldsymbol{\tau} \tag{10}$$

Step 2 — Internal energy (inherited, solved for e):

Compressible energy equation (`fluid` base)

$$\frac{\partial(\rho e)}{\partial t} + \nabla\cdot(\varphi\,e) + \nabla\cdot(\varphi\,K) + \nabla\cdot\left(\varphi\,\frac{p}{\rho}\right) + \nabla\cdot\mathbf{q} = S_e \tag{11}$$

Here $\varphi = \rho\mathbf{U}\cdot\mathbf{S}_f$ $[\mathrm{kg\,s^{-1}}]$ is the face mass flux, $K = \frac{1}{2}|\mathbf{U}|^2$ is specific kinetic energy, $\mathbf{q} = -\kappa\nabla T$ is the Fourier heat flux, and e $[\mathrm{J\,kg^{-1}}]$ is the sensible internal energy. After solving, the temperature is recovered from the equation of state via $T = T_{\mathrm{ref}} + e/C_v$ (for `eConst`) and density is updated.

Step 3 — Species concentration:

Species equation — `compressibleSpeciesFluid`

$$\frac{\partial C}{\partial t} + \nabla\cdot(\mathbf{U}_{\mathrm{vol}}\,C) = \nabla\cdot(D(T)\,\nabla C) - \frac{\partial C_t}{\partial t} + S_{\mathrm{vol}} \tag{12}$$

The volumetric flux $\mathbf{U}_{\mathrm{vol}}$ is derived from the mass flux:

$$\varphi_{\mathrm{vol}} = \frac{\varphi}{\rho_f} = \frac{\phi}{\texttt{fvc::interpolate(rho)}} \quad [\mathrm{m^3\,s^{-1}}] \tag{13}$$

Critical: the mass flux $\varphi\,[\mathrm{kg\,s^{-1}}]$ cannot be used directly in the species equation because C is molar concentration $[\mathrm{mol\ m^{-3}}]$, not mass concentration. Using φ would introduce a spurious factor of ρ in the advection term. The conversion $\varphi_{\mathrm{vol}} = \varphi/\rho_f$ is mandatory.

Thermophysical property format (`heRhoThermo`)

Unlike `speciesFluid`, which reads a flat `physicalProperties` dictionary, `compressibleSpeciesFluid` requires the full OpenFOAM `thermoType` hierarchy in `constant/<region>/physicalProperties`:

```
thermoType {
    type            heRhoThermo;
    mixture         pureMixture;
    transport       const;
    thermo          eConst;
    equationOfState rhoConst;   % or perfectGas for true compressibility
    specie          specie;
    energy          sensibleInternalEnergy;
}
mixture {
    specie          { molWeight  <M [g/mol]>; }
    equationOfState { rho  <rho [kg/m3]>; }    % for rhoConst
    thermodynamics  { Cv  <Cv [J/kg/K]>;  Hf  0; }
    transport       { mu  <mu [Pa.s]>;  Pr  <Pr>; }
}
```

A second dictionary `constant/<region>/thermophysicalTransport` is also required:

```
laminar { model Fourier; }
```

Required `fvSchemes` and `fvSolution`

fvSchemes — three additional `divSchemes` entries compared to `speciesFluid`:

```
divSchemes {
    div(phi,e)          Gauss linearUpwind grad(e);
    div(phi,K)          Gauss linear;
    div(phi,(p|rho))    Gauss linear;
    div(phiU,C_H2)      Gauss upwind;   % phiU = phi/rho_face
}
```

fvSolution — energy field is `e` (not `T`); density is updated explicitly via the EOS:

```
solvers {
    "e.*"   { solver PBiCGStab; preconditioner DILU;
              tolerance 1e-10; relTol 0; }
    "C_.*"  { solver PBiCGStab; preconditioner DILU;
              tolerance 1e-10; relTol 0; }
    "rho.*" { solver diagonal; }      % covers rhoFinal on final corrector
}
```

Pressure uses thermodynamic dimensions [Pa] (not kinematic $[\mathrm{m^2\ s^{-2}}]$): `dimensions [1 -1 -2 0 0 0 0]`, with `fixedValue` at the outlet (e.g. 101325 Pa for atmospheric reference).

Demonstrated by: `case329` (FLiBe molten-salt channel at 873 K with distributed H_2 source and SS316 permeation; `rhoConst` EOS validates equivalence to the incompressible `speciesFluid`), `case330` (He gas channel at 700 K, 1 atm; same source, D, and K_s as `case329` for cross-solver flux comparison).

Species and energy coupling — multi-region

In multi-region cases, the PIMPLE loop exchanges temperature across coupled patches via `coupledTemperature` boundary conditions (standard OpenFOAM), and exchanges species concentration via the custom `sievertsCoupledMixed` or `surfaceRecombination` BCs described in §4. Both exchanges happen every PIMPLE outer corrector, so the fields remain thermodynamically consistent throughout the time step.

Material Models

Arrhenius temperature dependence

Every material property that depends on temperature follows the Arrhenius form. This applies uniformly to D, the solubility K_s, and the surface-kinetics coefficients K_d, K_r:

Arrhenius property

$$X(T) = X_0 \exp\left(-\frac{E_a}{RT}\right) \tag{14}$$

Symbol	Meaning	Notes
X_0	Pre-exponential factor	Carries the SI units of X
E_a	Activation energy	J mol^{-1}
R	Gas constant	8.314 J mol^{-1} K^{-1}
T	Local temperature	K

Setting $E_a = 0$ gives a temperature-independent constant, used in all isothermal tutorial cases.

Dictionary syntax (applies to all Arrhenius properties):

```
D { X0  [0 2 -1 0 0 0 0]  2.9e-7;   Ea  13800; }
```

Isothermal ($E_a = 0$): `case311`–`case315`, `case317`–`case319`, `case321`–`case322`, `case328`–`case330`.
Arrhenius $D(T)$ active: `case316` (vapour diffusion), `case319` (WC and SS316), `case320` (SS316 in HX), `case323`–`case327` (PTFE membrane and/or SS316 wall).

Solubility laws

Sieverts' law (metals and interstitial solids)

Sieverts' law describes the equilibrium solubility of diatomic gases (H_2, D_2, T_2) that dissociate upon dissolution. The dissolved concentration is proportional to the *square root* of the gas partial pressure:

Sieverts' law

$$C = K_s(T)\sqrt{p} \tag{15}$$

$K_s(T)$ [mol m^{-3} Pa$^{-1/2}$] is the Sieverts solubility constant, following the Arrhenius form (14). Applicable materials include iron, stainless steel (SS316), tungsten, vanadium alloys, and palladium [6].

Used in: `case312`–`case314`, `case318`–`case320`, `case321` (metal layer), `case322`, `case323` (FLiBe and SS316 walls), `case329`–`case330` (fluid–SS316 Sieverts interface).

Henry's law (polymers, liquids, porous media)

For species that dissolve without dissociation (vapour in a polymer, dissolved gas in a liquid), the equilibrium concentration is proportional to pressure:

Henry's law

$$C = K_H(T)\, p \tag{16}$$

$K_H(T)$ [mol m^{-3} Pa^{-1}] is the Henry constant. Applicable to dense polymer films, membrane distillation membranes, and organic solvents [5].

Used in: `case315`–`case316` (membrane distillation), `case317` (reverse osmosis, $K_H = 0.5$ at the feed/membrane interface), `case321` (polymer layer).

McNabb–Foster trapping kinetics

In metals, hydrogen isotopes can occupy lattice defects (vacancies, dislocations, grain boundaries). The McNabb–Foster model [2] treats a single population of trap sites with density $n_t = f_t N$ [mol m^{-3}]:

McNabb–Foster ODE

$$\frac{\partial C_t}{\partial t} = \frac{\alpha_t}{N}\, C\,(n_t - C_t) \;-\; \alpha_d(T)\, C_t \tag{17}$$

Symbol	Meaning	Units
N	Molar trap-site density	mol m^{-3}
f_t	Trap fraction ($0 < f_t \leq 1$)	—
$n_t = f_t N$	Available trap-site density	mol m^{-3}
α_t	Trapping rate coefficient	s^{-1}
$\alpha_d(T) = \alpha_{d0} \exp(-\varepsilon / k_B T)$	Detrapping rate	s^{-1}
ε	Trap binding energy	eV
k_B	Boltzmann constant	J K^{-1}

The trapping term $\partial C_t / \partial t$ is subtracted from the mobile species equation (1) as a sink.

Trapping regimes

Two limiting regimes exist depending on the ratio of trapping to diffusion time scales:

- **Weak trapping** ($\alpha_t / D \ll 1$): traps fill slowly; the mobile front propagates at nearly the bare diffusion speed; breakthrough time $\approx L^2/D$.
- **Strong trapping** ($\alpha_t / D \gg 1$): most atoms are trapped; effective diffusivity $D_{\text{eff}} = D/(1 + N_t\, \alpha_t)$ is greatly reduced; breakthrough is delayed by orders of magnitude.

Numerical discretisation of the trapping ODE

The ODE (17) is stiff and must be discretised carefully to remain bounded ($0 \le C_t \le n_t$). A per-cell semi-implicit Euler step is used:

Semi-implicit Euler for trapping ODE

$$C_t^{n+1} = \frac{C_t^n + \Delta t \, \dfrac{\alpha_t}{N} C \, n_t}{1 + \Delta t \left(\dfrac{\alpha_t}{N} C + \alpha_d \right)} \tag{18}$$

This scheme is unconditionally stable and guarantees $0 \le C_t^{n+1} \le n_t$ for any time step.

Demonstrated by: `case313`. Two sub-cases span the weak and strong trapping regimes. The analytical breakthrough time is used to validate both limits.

Interface and Surface Boundary Conditions

Overview

At every coupled patch, two physical conditions must hold simultaneously:

1. **Flux continuity:** $D_A \, (\partial C_A/\partial n)_{\text{face}} = D_B \, (\partial C_B/\partial n)_{\text{face}}$
2. **Thermodynamic equilibrium:** determined by the solubility law on each side (§3.2).

These are implemented as a *mixed* (Robin) boundary condition in OpenFOAM, derived from the conjugate heat-transfer analogy.

Sieverts–Sieverts interface (linear partition)

When both sides follow Sieverts' law, the equilibrium condition at the interface face is:

Sieverts–Sieverts partition (`partition linear`)

$$\frac{C_A}{K_{s,A}} = \frac{C_B}{K_{s,B}} \quad \Longleftrightarrow \quad C_A = \frac{K_{s,A}}{K_{s,B}} C_B \tag{19}$$

The concentration ratio $K_{s,A}/K_{s,B}$ can be greater or less than unity, producing a concentration jump at the interface even in steady state. When $K_{s,A} = K_{s,B}$, the concentration is continuous.

Mixed-BC coefficients (self = region A, neighbour = region B):

$$\begin{aligned} \text{refValue} &= \frac{K_{s,A}}{K_{s,B}} C_{B,\text{cell}} \\ w &= \frac{D_B \, \delta_B^{-1}}{D_B \, \delta_B^{-1} + D_A \, \delta_A^{-1} (K_{s,A}/K_{s,B})} \\ C_{A,\text{face}} &= w \cdot \text{refValue} + (1 - w) \, C_{A,\text{cell}} \end{aligned} \tag{20}$$

where δ^{-1} is the distance from the cell centre to the interface face.

Demonstrated by: `case314` (two-layer composite membrane, K_s ratio = 4; interface concentration jump of factor 4 at steady state), `case318` (code-to-code benchmark, equal K_s, continuous profile), `case319` (WC/SS316 barrier, large K_s ratio responsible for PRF), `case320` (SS316 wall; also `case321` reference sub-case).

Henry–Sieverts interface (nonlinear partition)

When a Henry-law material (polymer, liquid) interfaces with a Sieverts-law material (metal), the same partial pressure p_{int} must hold on both sides:

$$p_{\text{int}} = \frac{C_{\text{Henry}}}{K_H} = \left(\frac{C_{\text{Sieverts}}}{K_s}\right)^2$$

This gives two equivalent partition conditions depending on which side is considered "self":

Henry–Sieverts partition

$$\text{Sieverts side (}\texttt{partition sqrt}\text{):} \quad C_{\text{Sieverts}} = K_s\sqrt{\frac{C_{\text{Henry}}}{K_H}} \tag{21}$$

$$\text{Henry side (}\texttt{partition quadratic}\text{):} \quad C_{\text{Henry}} = K_H\left(\frac{C_{\text{Sieverts}}}{K_s}\right)^2 \tag{22}$$

Both sides must be set consistently in the `0/` boundary field files; they encode the same physics from opposite perspectives.

Linearised mixed-BC weight for the nonlinear cases. The weight (20) is derived by linearising the partition function at the current Picard iterate. For the `sqrt` mode (self = Sieverts, neighbour = Henry):

$$\left.\frac{\mathrm{d}C_s}{\mathrm{d}C_n}\right|_{\text{iterate}} = \frac{C_{s,\text{face}}}{2\,C_{n,\text{cell}}} \qquad \Rightarrow \qquad \text{selfKD} = D_A\,\delta_A^{-1}\,\frac{C_{A,\text{face}}}{2\,C_{B,\text{cell}}} \tag{23}$$

Using the linearised derivative (rather than the constant K_s/K_H) in the weight formula ensures that flux continuity is enforced exactly at every Picard step, even in steady state on a coarse mesh.

Steady-state interface concentrations

For the case of equal layer thicknesses L, equal diffusivities D, inlet concentration $C_0 = 1$ mol m^{-3}, and $K_s = K_H = 1$:

Analytical interface concentrations (equal D, L, K)

$$u^2 + u = 1, \quad u = C_{\text{Sieverts,int}}, \qquad \Rightarrow \qquad u = \tfrac{\sqrt{5}-1}{2} \approx 0.618 \text{ mol m}^{-3} \tag{24}$$

The *golden-ratio conjugate* $(\sqrt{5}-1)/2$ appears naturally because the steady-state flux equality combined with the $\sqrt{\cdot}$ partition gives exactly the quadratic $u^2 + u - 1 = 0$. The Henry (polymer) side holds $C_{\text{Henry,int}} = u^2 \approx 0.382$ mol m^{-3}, demonstrating that the *concentration is discontinuous and higher on the Sieverts (metal) side* — the defining signature of this partition law.

The steady-state flux in the Henry–Sieverts case is 23.6 % *higher* than the equivalent Sieverts–Sieverts case with the same D, L, and K:

$$J_{\text{HS}} = D\,\frac{C_{\text{Sieverts,int}}}{L} = \frac{(\sqrt{5}-1)/2}{L/D}\,C_0 \;\approx\; 1.236\,J_{\text{SS}} \tag{25}$$

Demonstrated by: `case321`. Two sub-cases (`sieverts_sieverts` reference and `henry_sieverts`) run with identical D, L, boundary values; the 23.6 % flux difference is confirmed analytically and numerically.

Surface recombination and dissociation

At a free metal surface exposed to a molecular gas, two kinetic processes compete:

Dissociative absorption:	$H_2 \rightarrow 2H$,	flux $= K_d(T)\, p_{\text{gas}}$
Recombinative desorption:	$2H \rightarrow H_2$,	flux $= K_r(T)\, C^2$

The net flux into the solid is the *surface recombination Robin BC*:

Surface recombination BC (`surfaceRecombination`)

$$D\,\frac{\partial C}{\partial n} \;=\; K_d(T)\, p_{\text{gas}} \;-\; K_r(T)\, C^2 \tag{26}$$

Symbol	Meaning	Units
$K_d(T)$	Dissociation rate (Arrhenius)	mol m^{-2} s^{-1} Pa^{-1}
$K_r(T)$	Recombination rate (Arrhenius)	m^4 mol^{-1} s^{-1}
p_{gas}	Gas-phase partial pressure	Pa

Both K_d and K_r follow the Arrhenius form (14). Setting $p_{\text{gas}} = 0$ models a vacuum or purge side (pure desorption).

Equilibrium and Sieverts limit

At equilibrium (zero net flux), $C = C_{\text{eq}}$:

$$K_d\, p_{\text{gas}} = K_r\, C_{\text{eq}}^2 \qquad \Rightarrow \qquad C_{\text{eq}} = \sqrt{\frac{K_d\, p_{\text{gas}}}{K_r}} \tag{27}$$

This is exactly Sieverts' law (15) with $K_s = \sqrt{K_d/K_r}$: *the surface recombination BC generalises the Sieverts equilibrium BC and recovers it in the fast-kinetics limit* [3].

Damköhler number

The Damköhler number compares the surface-kinetics rate to the bulk-diffusion rate:

Damköhler number

$$\text{Da} \;=\; \frac{K_r\, C_{\text{eq}}\, L}{D} \tag{28}$$

Da	Limiting step	Surface concentration
$\text{Da} \gg 1$	Diffusion-limited	$C_s \approx C_{\text{eq}}$ (Sieverts equilibrium)
$\text{Da} \approx 1$	Mixed control	$C_s = \frac{\sqrt{5}-1}{2}\, C_{\text{eq}}$ (see below)
$\text{Da} \ll 1$	Kinetically limited	$C_s \ll C_{\text{eq}}$

Steady-state surface concentration

At steady state, the Robin condition (26) must equal the permeation flux $D\, C_s/L$ (assuming zero concentration at the purge face):

$$K_r\, C_s^2 + \frac{D}{L}\, C_s - K_d\, p_{\text{gas}} = 0 \tag{29}$$

The physical (positive) root is:

Steady-state surface concentration (general Da)

$$C_s = \frac{-D/L + \sqrt{(D/L)^2 + 4\,K_r\,K_d\,p_{\text{gas}}}}{2\,K_r} \tag{30}$$

Special case $\text{Da} = 1$ (with $K_d = K_r$, $p_{\text{gas}} = 1$, $D/L = K_r\,C_{\text{eq}} = K_r$):

$$C_s^2 + C_s - 1 = 0 \qquad \Rightarrow \qquad C_s = \tfrac{\sqrt{5}-1}{2} \approx 0.618 \text{ mol m}^{-3} \tag{31}$$

The golden-ratio conjugate appears again. The flux is reduced by 38.2 % relative to the Sieverts equilibrium limit:

$$\frac{J_{\text{surf}}}{J_{\text{ref}}} = \frac{C_s}{C_{\text{eq}}} = \frac{\sqrt{5}-1}{2} \approx 0.618 \tag{32}$$

Numerical treatment. The C^2 term is Picard-frozen at the previous cell concentration, making the gradient a simple field evaluation each PIMPLE outer iteration. Convergence is guaranteed by the PIMPLE outer loop.

Demonstrated by: `case322`. Sub-case `sieverts_bc` provides the $\text{Da} \to \infty$ reference (J_{ref}); sub-case `surface_recombination` runs $\text{Da} = 1$ and confirms $C_s = (\sqrt{5}-1)/2$ and the 38.2 % flux reduction.

Performance Metrics

Permeation flux and speciesFlux function object

The area-integrated molar permeation flux through a patch is:

Surface molar flux

$$J_{\text{patch}} = -\int_{\text{patch}} D\,\nabla C \cdot \hat{n}\,\mathrm{d}A \qquad [\text{mol s}^{-1}] \tag{33}$$

The `speciesFlux` function object evaluates this integral at every write time without modifying the solver. It outputs both the area-averaged flux density [mol m^{-2} s^{-1}] and the total flux [mol s^{-1}] to `postProcessing/<name>/<time>/speciesFlux.dat`.

The diffusivity field `D_<species>` is registered as `AUTO_WRITE` by the `speciesModel`, making it available to the function object at every write time.

Dictionary syntax:

```
functions
{
    wallPermeation
    {
        type        speciesFlux;
        libs        ("libspeciesPost.so");
        region      wall;
        species     C_H2;
        patches     (wall_to_hitec);
        writeControl writeTime;
    }
}
```

Demonstrated by: `case320` (permeation flux through SS316 wall into Hitec coolant), `case321`, `case322` (permeation flux at purge face).

Permeation reduction factor

The permeation reduction factor (PRF) quantifies the effectiveness of a barrier coating:

Permeation reduction factor

$$\mathrm{PRF} = \frac{J_{\text{without barrier}}}{J_{\text{with barrier}}} = \frac{C_0/R_{\text{total,bare}}}{C_0/R_{\text{total,coated}}} \tag{34}$$

For a two-layer composite (coating thickness L_c, substrate thickness L_s) with equal K_s on both sides, the steady-state PRF can be approximated by the ratio of total diffusion resistances. High PRF arises when the coating has a much larger K_s/D ratio (lower permeability) than the substrate.

The permeability of a material is $\Phi = K_s^2 D$ (in $[\,\mathrm{mol\ m^{-1}\ s^{-1}\ Pa^{-1}}]$, the Richardson–Dushman formula for diatomic gases).

Demonstrated by: `case319`. A tungsten carbide (WC) coating on SS316 at $T = 1073$ K:

$$\mathrm{PRF} \approx 200$$

confirmed by comparing the `with_coating` and `no_coating` sub-cases. The PRF is dominated by the large K_s ratio ($K_{s,\mathrm{WC}} \gg K_{s,\mathrm{SS316}}$), which lowers the dissolved concentration in the coating and therefore reduces the flux into the substrate.

Membrane Equilibrium Boundary Conditions

Two additional boundary conditions support membrane distillation modelling where the species concentration and the thermal boundary condition at the membrane face are coupled through the vapour pressure.

Antoine equilibrium (`antoineEquilibrium`)

The `antoineEquilibrium` boundary condition is a `fixedValueFvPatchScalarField` that computes the equilibrium vapour molar concentration from the local face temperature using the Antoine equation:

Antoine vapour pressure

$$\ln(p_{\text{vap}}\,[\mathrm{Pa}]) = A - \frac{B}{C_{\text{ant}} + T} \tag{35}$$

The face concentration is set using the ideal-gas law:

$$C_{\text{face}} = \frac{p_{\text{vap}}}{R\,T} \qquad [\,\mathrm{mol\ m^{-3}}] \tag{36}$$

The temperature T is read from the registered temperature field on the same patch. Because T at the membrane face changes as the flow solution converges, `antoineEquilibrium` is evaluated every PIMPLE outer corrector, providing a dynamic coupling between concentration and temperature.

Dictionary syntax:

```
C_H2O_at_membrane
{
    type        antoineEquilibrium;
    A           23.197;    // ln(Pa) form of Antoine equation
    B           3885.7;    // K
    C           -42.98;    // K  (negative shifts reference)
    value       uniform 0;
}
```

This BC replaces `codedFixedValue` used in earlier cases and is physically identical but compiled and registered as a proper named BC.

Demonstrated by: `case325`, `case326`, `case327` (membrane face C_{H_2O}, both membrane–feed and membrane–permeate interfaces).

Latent heat flux (`latentHeatFlux`)

The `latentHeatFlux` boundary condition is a `mixedFvPatchScalarField` applied to the temperature field T in fluid channel regions at their membrane-facing patch. It adds the latent heat of phase change to the thermal balance:

Latent heat coupling

$$q_{\text{latent}} = J \cdot L_{\text{vap}} \cdot M \qquad [\text{W m}^{-2}] \tag{37}$$

where $J\,[\text{mol m}^{-2}\,\text{s}^{-1}]$ is the vapour molar flux through the membrane (computed from the neighbour region's diffusivity and concentration gradient), $L_{\text{vap}}\,[\text{J mol}^{-1}]$ is the molar latent heat, and $M\,[\text{kg mol}^{-1}]$ is the molar mass.

The BC contributes q_{latent} to the heat balance as a `refGrad` correction: on the evaporation (feed) side, heat is *removed* by evaporation; on the condensation (permeate) side, heat is *added*. The value fraction follows the same conjugate-transfer formula as `coupledTemperature`, ensuring flux continuity for the thermal field.

Dictionary syntax:

```
feed_at_membrane
{
    type        latentHeatFlux;
    Lvap        44000;          // J/mol  (water at 60°C)
    M           0.018015;       // kg/mol
    CName       C_H2O;          // concentration field driving the flux
    side        evaporation;    // or: condensation
    value       uniform 333;
}
```

Demonstrated by: `case326`, `case327` (replaces `coupledTemperature` at membrane faces in feed and permeate regions).

Solver Selection Guide

Choosing between `speciesSolid` and `speciesFluid` for a fluid channel region is the most important design decision in setting up a new case. The following criteria apply.

Use `speciesSolid` for a fluid region when

- **The velocity field is known in advance.** Plug flow ($\mathbf{U} = \text{const}$), an analytically prescribed Poiseuille profile, or a uniform-bulk-velocity approximation. Set a `fixedValue` (or `uniformFixedValue`) **U** field in `0/<region>/U` and add the appropriate `div(phi,C_<species>)` and `div(rhoPhi,e)` entries to `fvSchemes`.

- **The primary research question is species or heat transport, not flow development.** When advection is simply a forcing term and its precise spatial distribution is not critical, the analytical-velocity approximation reduces cost and complexity without meaningful loss of accuracy.

- **The effective diffusivity already encodes flow effects.** In turbulent channels, the turbulent diffusivity $D_{\text{eff}} \approx \nu_t/\text{Sc}_t$ can be orders of magnitude larger than the molecular value. Using a constant D_{eff} in `speciesSolid` captures the bulk transport without resolving the velocity field (e.g. case323, $D_{\text{eff}} \approx 10^{-4}$ m^2 s^{-1}).

- **The region is a solid.** `speciesSolid` uses `solidThermo` for temperature, which is the correct formulation for metallic walls, membranes, and barriers. No momentum equation is required.

Use `speciesFluid` for a fluid region when

- **The velocity profile is unknown and must be computed.** Developing flow (entry region), complex geometry, or pressure-driven flow in a channel of arbitrary cross-section where the parabolic/ turbulent profile is not prescribed.

- **Concentration polarisation from a developing boundary layer must be captured.** At high Schmidt number ($\text{Sc} = \nu/D \gg 1$), the concentration boundary layer at the permeation wall is much thinner than the momentum BL. Resolving $\delta_C \sim L\,\text{Pe}^{-1/3}$ requires the correct velocity profile from the solved PIMPLE loop.

- **The pressure field is physically meaningful.** Cases where the pressure drop, velocity profile, or flow recirculation influences the result require the full incompressible Navier–Stokes solution.

- **The energy equation involves convective transport.** `speciesSolid` uses the `solidThermo` internal energy form $\partial(\rho e)/\partial t + \nabla \cdot \mathbf{q}$, which is appropriate for solids. For a fluid, the temperature advection term $\nabla \cdot (\mathbf{U}\,T)$ is essential; `speciesFluid` provides it via the scalar transport form (5).

Use `compressibleSpeciesFluid` for a fluid region when

- **Density varies significantly with temperature or pressure.** Gases with large ΔT across the channel (e.g. helium at 700 K, 1 atm or FLiBe molten-salt at 873 K) require a proper compressible energy equation where pressure-work, kinetic-energy flux, and variable density all couple to the velocity field.

- **The equation of state is non-trivial.** `compressibleSpeciesFluid` uses `heRhoThermo` with user-specified EOS (`rhoConst`, `perfectGas`, etc.), allowing the density–temperature–pressure coupling to be resolved consistently.

- **Sensible internal energy (rather than temperature) must be the primary energy variable.** The `fluid` base solves the full compressible energy equation in terms of e (11),

including the pressure-work term $\nabla \cdot (\varphi\, p/\rho)$. The temperature T is then recovered via the EOS; this avoids operator-splitting errors that arise when T is used as the primary variable in a compressible flow.

- **Cross-solver validation against `speciesFluid` is needed.** When two different thermophysical frameworks (incompressible vs. compressible) should give the same species flux under equivalent conditions, running both and comparing the permeation flux is a strong verification tool (see `case329`–`case330`).

Rule of thumb. For laminar channel flow at Pe $\lesssim 100$, advection and diffusion compete on similar length scales and the velocity profile matters. Use `speciesFluid` (incompressible) or `compressibleSpeciesFluid` (variable-density gas or liquid). For Pe $\gg 100$ in engineering contexts where only the bulk flux is needed, a turbulent-effective-D in `speciesSolid` is usually sufficient and much cheaper.

Numerical Implementation

Finite-volume discretisation

The species equation (1) is discretised using the standard OpenFOAM finite-volume method on an unstructured collocated mesh:

- **Time integration:** first-order implicit Euler (`ddtSchemes: Euler`).
- **Diffusion:** second-order Gauss linear with explicit non-orthogonal correction (`laplacianSchemes: Gauss linear corrected`).
- **Gradient:** Gauss linear (`gradSchemes: Gauss linear`).

These settings are appropriate for diffusion-dominated transport on orthogonal structured meshes (all tutorial cases use 1-D block meshes).

PIMPLE outer loop and Picard convergence

The nonlinear boundary conditions (`quadratic`, `sqrt`, `surfaceRecombination`) are linearised by Picard (fixed-point) iteration within the PIMPLE outer corrector loop:

1. **Outer iteration k:** the nonlinear BC evaluates the interface condition using cell values from iteration $k-1$ (frozen coefficients).
2. **Species equation:** assembled and solved with the frozen coefficients; cell values are updated.
3. **Convergence check:** if the species residual $< \varepsilon$ (`residualControl`), the outer loop exits; otherwise another outer iteration is performed.

The mixed-BC weight (23) uses the *linearised derivative* of the partition function evaluated at the current iterate, ensuring that flux continuity is enforced at every Picard step and that the method converges to the exact partition condition as $k \to \infty$.

Coupled-patch data exchange

At each outer PIMPLE iteration, the coupled patches exchange:

- **Neighbour cell concentration** $C_{n,\text{cell}}$ (used to evaluate the partition refValue),

- **Neighbour diffusivity** D_n and **inverse distance** δ_n^{-1} (used to evaluate nbrKD for the weight).

Data exchange uses OpenFOAM's mapped-patch infrastructure with an incremented MPI message tag to avoid collisions when multiple coupled fields are exchanged in the same iteration.

Semi-implicit trapping ODE

The McNabb–Foster ODE (17) is solved *cell by cell* with the per-cell semi-implicit Euler update (18). This is performed explicitly before the species equation assembly, so the sink term $\partial C_t/\partial t$ is available for the right-hand side. No global matrix inversion is required for the ODE update.

Library architecture

Library	Purpose	Key classes
`libspeciesTransport.so`	Core physics	`speciesModel`, `arrheniusProperty`, `McNabbFoster`, `noTrapping`
`libspeciesCoupling.so`	Boundary conditions	`sievertsCoupledMixed`, `surfaceRecombination`, `antoineEquilibrium`, `latentHeatFlux`
`libspeciesSolid.so`	Solver: solid / prescribed-flow	`speciesSolid`
`libspeciesFluid.so`	Solver: incompressible fluid	`speciesFluid`
`libcompressibleSpeciesFluid.so`	Solver: compressible fluid	`compressibleSpeciesFluid`
`libspeciesPost.so`	Post-processing	`speciesFlux`

All cases must list the required libraries in `system/controlDict`. Cases using only solid regions:

```
libs ("libspeciesTransport.so" "libspeciesCoupling.so"
      "libspeciesSolid.so"    "libspeciesPost.so");
```

Cases with one or more `speciesFluid` regions also add:

```
libs (...  "libspeciesFluid.so");
```

Cases with one or more `compressibleSpeciesFluid` regions use instead:

```
libs (...  "libcompressibleSpeciesFluid.so");
```

Tutorial Guide

Twenty tutorial cases cover the full range from pure-diffusion verification to fully coupled compressible fluid-flow species transport. Cases 311–322 use `speciesSolid` exclusively; cases 323–326 extend `speciesSolid` with prescribed convection; cases 327–328 introduce `speciesFluid`; cases 329–330 use `compressibleSpeciesFluid`.

Each case contains an `Allrun` script, an `Allclean` script, a per-case `README.md`, and (for verification cases) a Python validation script in `validation/` that compares the simulation output to an analytical or benchmark solution (exit code 0 on pass, 1 on fail).

Case summary table

Table 1: Tutorial cases and the physics they demonstrate.

Case	Solver(s)	Physics / Key feature
case311	solid	Pure 1-D diffusion, step BC. Temperature field used as spec proxy. Validates erfc front propagation (exact analytical s tion).
case312	speciesSolid	1-D diffusion, non-uniform IC (setFields). Fourier cos series analytical solution. First use of the species equation.
case313	speciesSolid	McNabb–Foster trapping (§3.3). Weak ($\alpha_t/D \ll 1$) a strong ($\alpha_t/D \gg 1$) trapping regimes. Validates semi-impl ODE (18).
case314	speciesSolid	Two-region composite membrane; Sieverts–Sieverts linear p tition (§4.2), $K_{s,A}/K_{s,B} = 4$. Validates interface concentrat jump.
case315	speciesSolid	Isothermal membrane distillation; Henry-law solubility, c stant D. Linear steady-state profile.
case316	speciesSolid	Membrane distillation with Arrhenius $D(T)$. Temperat gradient drives non-linear steady-state profile. Full ener species coupling.
case317	speciesSolid	Reverse osmosis: two-region feed/membrane. Henry partit at feed face ($K_H = 0.5$). Solution-diffusion model.
case318	speciesSolid	Code-to-code benchmark vs Pasler et al. and Fourier sine ser Constant BCs; transient diffusion; exact analytical solution
case319	speciesSolid	WC coating + SS316; real Arrhenius $D(T)$ for both materi PRF ≈ 200 (§5.2). Two sub-cases with/without coating.
case320	speciesSolid	Three-region shell-tube HX: FLiBe | SS316 | Hitec. Conjug heat + H_2 permeation; Arrhenius $D(T)$. speciesFlux measures J.
case321	speciesSolid	Henry–Sieverts partition (§4.3). Two sub-ca sieverts_sieverts and henry_sieverts. Valida 23.6 % flux increase; golden-ratio interface (24).
case322	speciesSolid	Surface recombination / dissociation (§4.4). Da= 1: C_s $(\sqrt{5}-1)/2$; 38.2 % flux reduction vs Sieverts BC.
case323	speciesSolid	2-D counter-flow FLiBe–SS316–Hitec HX. Prescribed turbul plug flow in fluid channels via registered **U** field. Turbule effective D_{eff} absorbs flow effects.
case324	speciesSolid	DCMD validation vs Khalifa 2017. Single-region PT membrane; membrane face C set from Antoine equation fixedValue. $J_{\text{sim}} = 31.1$ vs $J_{\text{exp}} = 35\,\text{L}\,\text{m}^{-2}\,\text{h}^{-1}$.
case325	speciesSolid	DCMD with CFD temperature polarisation. Three-region 2 prescribed plug flow in feed/permeate; antoineEquilibr at membrane faces (§6.1). Temperature polarisation coeffici (TPC) emerges from coupled CFD rather than a heat-trans correlation.
case326	speciesSolid	DCMD with latent heat coupling. latentHeatFlux BC (§ replaces coupledTemperature at membrane faces; adds $J\,L_{\text{vap}}\,M$ to the thermal balance. Full membrane-distillat energy accounting.
case327	speciesFluid + speciesSolid	DCMD with solved incompressible flow. Feed/permea speciesFluid (PIMPLE **U**/p + scalar T + C). Membra speciesSolid. Demonstrates the complete speciesFlu workflow with antoineEquilibrium and latentHeatFlux coupled faces.
case328	speciesFluid	Solver validation: single-region 2-D channel, water at R 20. Three quantitative checks: Poiseuille profile ($L_2 < 2$ pressure gradient ($< 5\,\%$), Graetz $\text{Nu}_\infty = 7.5407$ ($< 5\,\%$).

continued on next page

Case	Solver(s)	Physics / Key feature
`case329`	`compressibleSpeciesFluid` + `speciesSolid`	FLiBe molten-salt channel (873 K) with distributed H_2 sou → SS316 permeation. `heRhoThermo/rhoConst` EOS; compr ible PIMPLE; Pe = 1000; Sieverts linear interface; vacu outer BC. Validates equivalence of compressible and inc pressible formulations for constant-density liquids.
`case330`	`compressibleSpeciesFluid` + `speciesSolid`	He gas channel (700 K, 1 atm) with distributed H_2 source SS316 permeation. Identical D, K_s, source, and inlet velocity `case329`. Cross-solver validation: confirms that the compr ible solver gives the same permeation flux as the rhoConst-liq case across a 10× change in density.

Physics-to-tutorial cross-reference

Table 2: Which tutorials exercise which physical models (all 20 cases). ✓ = feature present; blank = not applicable.

Physical model	*311*	*312*	*313*	*314*	*315*	*316*	*317*	*318*	*319*	*320*	*321*	*322*	*323*	*324*	*325*	*326*	*327*	*328*	*329*	*330*
Species diffusion eq. (1)	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Energy eq. (2)	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Arrhenius $D(T)$ (14)						✓			✓	✓			✓	✓	✓	✓	✓			
Sieverts law (15)			✓	✓				✓	✓	✓	✓	✓	✓						✓	✓
Henry law (16)					✓	✓	✓				✓			✓	✓	✓	✓			
McNabb–Foster (17)			✓																	
Linear partition (19)				✓				✓	✓	✓	✓		✓						✓	✓
Sqrt partition (21)											✓									
Quadratic partition (22)							✓				✓									
Surface recombination (26)												✓								
PRF metric (34)									✓											
speciesFlux FO (33)										✓	✓	✓	✓	✓	✓	✓	✓			
Multi-region ($\geq$ 2 regions)										✓			✓		✓	✓	✓		✓	✓
Antoine equilibrium BC (§6.1)														✓	✓	✓	✓			
Latent heat flux BC (§6.2)																✓	✓			
Incompressible PIMPLE (4)																	✓	✓		
Compressible PIMPLE (10)																		✓	✓	
Volumetric source S_{vol} (1)																			✓	✓

Running the tutorials

Prerequisites. Source OpenFOAM-13 and build the libraries:

```
source /opt/openfoam13/etc/bashrc
cd $WM_PROJECT_USER_DIR/multiSpeciesRegionFoam
./Allwmake 2>&1 | tee build.log
```

Individual case:

```
cd tutorials/case314-composite-membrane
./Allrun
python3 validation/membrane_compare.py --plot
```

Validation exit code: each validation script exits 0 on PASS (flux within 5 % of analytical) and 1 on FAIL.

Notation Summary

Symbol	Meaning	SI units
C	Mobile species concentration	mol m^{-3}
C_t	Trapped concentration	mol m^{-3}
$D(T)$	Diffusivity (Arrhenius)	m^2 s^{-1}
e	Sensible internal energy (`compressibleSpeciesFluid`)	J kg^{-1}
E_a	Activation energy	J mol^{-1}
f_t	Trap fraction	—
J	Molar flux (area-averaged)	mol m^{-2} s^{-1}
$K = \frac{1}{2}\lvert\mathbf{U}\rvert^2$	Specific kinetic energy (`compressibleSpeciesFluid`)	J kg^{-1}
$K_d(T)$	Dissociation rate	mol m^{-2} s^{-1} Pa^{-1}
$K_H(T)$	Henry solubility constant	mol m^{-3} Pa^{-1}
$K_r(T)$	Recombination rate	m^4 mol^{-1} s^{-1}
$K_s(T)$	Sieverts solubility constant	mol m^{-3} Pa$^{-1/2}$
L	Layer thickness	m
N	Molar trap-site density	mol m^{-3}
$n_t = f_t N$	Available trap density	mol m^{-3}
p	Pressure (thermodynamic in compressible, kinematic in incompressible)	Pa or m^2 s^{-2}
R	Gas constant (8.314)	J mol^{-1} K^{-1}
S_{vol}	Volumetric species source	mol m^{-3} s^{-1}
T	Temperature	K
$\mathbf{U}$	Fluid velocity	m s^{-1}
w	Mixed-BC value fraction	—
X_0	Arrhenius pre-exponential factor	(same as X)
$\alpha_d(T)$	Detrapping rate	s^{-1}
α_t	Trapping rate	s^{-1}
δ^{-1}	Cell-centre to face distance (inv.)	m^{-1}
ε	Trap binding energy	eV
$\kappa(T)$	Thermal conductivity	W m^{-1} K^{-1}
ρ	Fluid density	kg m^{-3}
φ	Face mass flux ($= \rho\mathbf{U}\cdot\mathbf{S}_f$, `phi` in OpenFOAM)	kg s^{-1}
φ_{vol}	Face volumetric flux ($= \varphi/\rho_f$, `phiU`)	m^3 s^{-1}
$\Phi = K_s^2 D$	Permeability (diatomic gas)	mol m^{-1} s^{-1} Pa^{-1}
Da	Damköhler number (surface kinetics)	—
PRF	Permeation reduction factor	—

References

[1] Hattab, N., Siriano, S., Giannetti, F. *An OpenFOAM multi-region solver for tritium transport modeling in fusion systems. Fusion Engineering and Design* **202** (2024) 114362.

[2] McNabb, A., Foster, P. K. *A new analysis of the diffusion of hydrogen in iron and ferritic steels. Trans. Metall. Soc. AIME* **227** (1963) 618–627.

[3] Pick, M. A., Sonnenberg, K. *A model for atomic hydrogen-metal interactions — application to recycling, recombination and permeation. J. Nucl. Mater.* **131** (1985) 208–220.

[4] Anderl, R. A. et al. *Hydrogen transport behavior of metal coatings for plasma-facing components. J. Nucl. Mater.* **196–198** (1992) 986–991.

[5] Baker, R. W. *Membrane Technology and Applications*, 3rd ed. Wiley, 2012.

[6] Ockwig, N. W., Nenoff, T. M. *Membranes for hydrogen separation. Chem. Rev.* **107** (2007) 4078–4110.

[7] CFD Direct. *Modular Solvers in OpenFOAM.* https://cfd.direct/openfoam/free-software/modular-solvers/